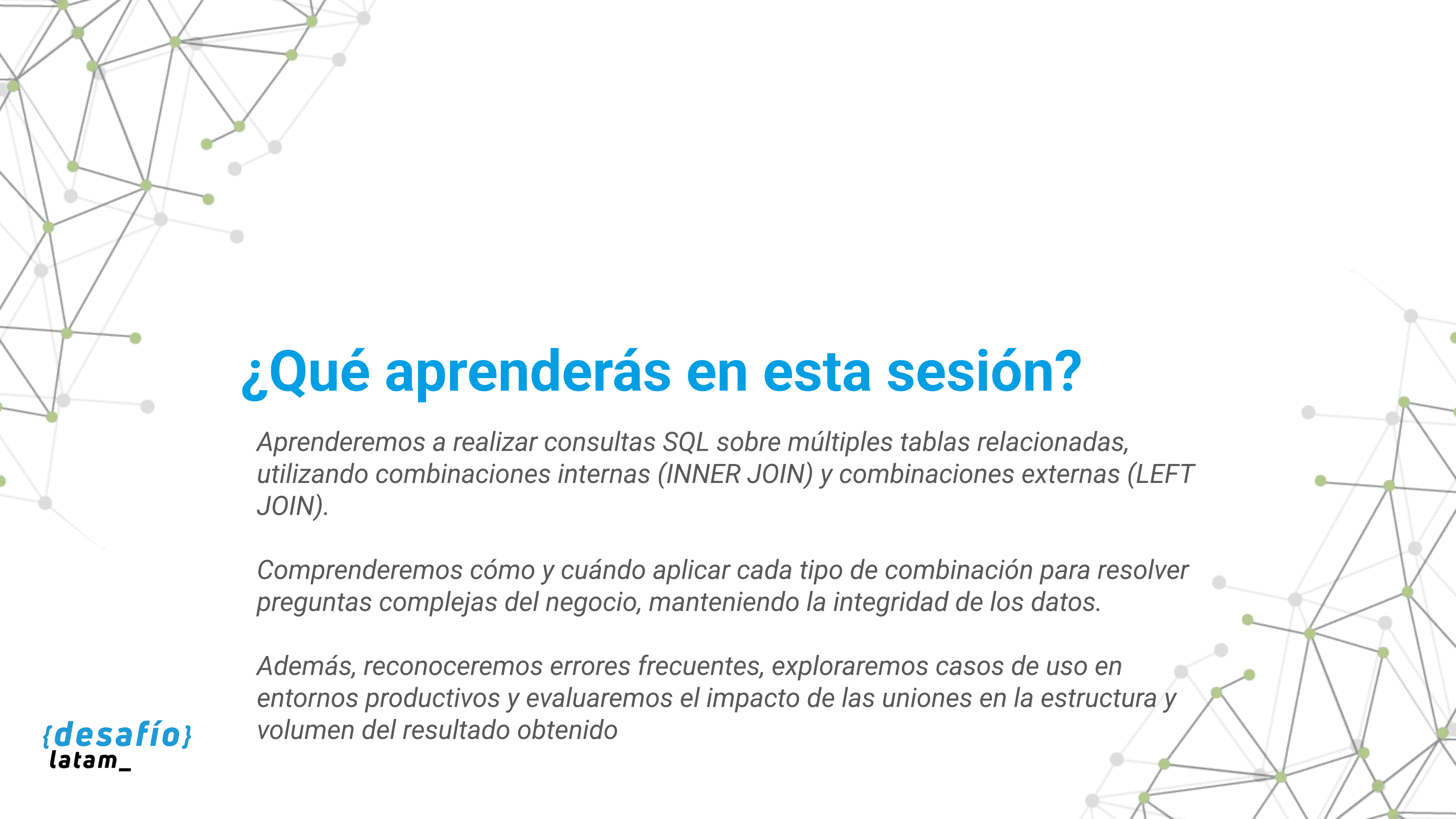




Consultas Avanzadas y Relaciones

**Clase 2 - Consultas de selección con tablas relacionadas
(Parte I)**



¿Qué aprenderás en esta sesión?

Aprenderemos a realizar consultas SQL sobre múltiples tablas relacionadas, utilizando combinaciones internas (INNER JOIN) y combinaciones externas (LEFT JOIN).

Comprenderemos cómo y cuándo aplicar cada tipo de combinación para resolver preguntas complejas del negocio, manteniendo la integridad de los datos.

Además, reconoceremos errores frecuentes, exploraremos casos de uso en entornos productivos y evaluaremos el impacto de las uniones en la estructura y volumen del resultado obtenido

Desafío inicial

Trabajas como analista de datos en una empresa de servicios digitales que gestiona contratos con distintos clientes y registra las órdenes de servicio de forma separada. Te piden generar un reporte mensual con los nombres de los clientes, sus datos de contacto y la cantidad de órdenes emitidas en el último mes.

Sin embargo, los datos están repartidos entre las tablas clientes y ordenes_servicio, las cuales comparten un campo en común llamado id_cliente. Sabes que no todos los clientes han emitido órdenes en el periodo y que algunos registros pueden estar incompletos o desalineados.

¿Cómo unir ambas tablas para obtener una visión integrada? ¿Qué ocurre si usas un INNER JOIN versus un LEFT JOIN? ¿Cómo decides cuál usar?

¿Qué precauciones debes tomar para que no se pierda información crítica?

Para responder a estas preguntas, necesitas dominar los conceptos de combinaciones entre tablas y evaluar correctamente sus consecuencias.

Combinaciones de tablas en SQL: contexto y necesidad

En el mundo real, los datos no están almacenados en una única tabla. Las bases de datos relacionales se estructuran en múltiples tablas con campos clave que permiten enlazarlas. Esta modularidad facilita la gestión, escalabilidad y seguridad de los datos, pero también requiere habilidades específicas para extraer información conjunta desde estas fuentes fragmentadas.

Cuando necesitas saber, por ejemplo, cuántos productos compró cada cliente, o cuál fue el último servicio solicitado por cada usuario, estás combinando información de distintas tablas. Ahí es donde entran en juego las combinaciones (JOINS).

SQL ofrece varios tipos de combinaciones. En esta sesión nos enfocaremos en los dos más utilizados en contextos productivos: INNER JOIN y LEFT JOIN. Estos permiten integrar tablas bajo diferentes condiciones de coincidencia.



INNER JOIN: unión de registros coincidentes

¿Qué es un INNER JOIN?

INNER JOIN permite combinar registros de dos o más tablas siempre que exista una coincidencia entre los valores de las columnas clave relacionadas. El resultado de la consulta solo incluirá las filas en las que haya coincidencia entre ambas tablas.

Sintaxis básica

```
SELECT
    c.nombre,
    o.fecha_orden
FROM clientes c
INNER JOIN ordenes o ON c.id_cliente = o.id_cliente;
```

Aplicación práctica

Este tipo de combinación se usa cuando sólo se requiere la información de registros que tienen correspondencia en ambas tablas. Por ejemplo, listar las órdenes que han sido emitidas, con sus respectivos clientes. Si un cliente no ha emitido órdenes, no aparecerá en el resultado.

Contexto productivo

En retail, una consulta con INNER JOIN puede ayudar a obtener todas las transacciones realizadas por clientes activos, descartando registros vacíos o sin operaciones.

Buenas prácticas y errores comunes

Buenas prácticas

- Usar alias para mejorar la legibilidad.
- Siempre identificar las llaves de relación con claridad.
- Evitar nombres ambiguos en columnas.

Errores comunes

- Omitir la cláusula ON, lo que genera un producto cartesiano.
- Relacionar campos de distinto tipo o con nombres similares pero no equivalentes.
- Asumir que todas las claves tienen coincidencias.

INNER JOIN es ideal para analizar relaciones sólidas donde se requiere precisión y solo se necesitan registros con coincidencias en ambas tablas.

Sin embargo, existen muchos escenarios donde se requiere conservar información, incluso si no hay relación directa. En ese caso, recurrimos a LEFT JOIN.



LEFT JOIN: conservación de todos los registros de una tabla

¿Qué es un LEFT JOIN?

LEFT JOIN (o combinación externa izquierda) permite unir dos tablas manteniendo todos los registros de la tabla de la izquierda, y sólo las coincidencias de la tabla de la derecha. Si no hay coincidencia, los campos de la segunda tabla se rellenan con NULL.

Sintaxis básica

```
SELECT
    c.nombre,
    o.fecha_orden
FROM clientes c
LEFT JOIN ordenes o ON c.id_cliente = o.id_cliente;
```

Aplicación práctica

Se utiliza cuando es importante mantener todos los registros de la tabla principal, incluso si no tienen correspondencia en la otra. Por ejemplo, obtener un listado completo de clientes y verificar si emitieron órdenes, aunque algunos no lo hayan hecho aún.

Contexto productivo

En una plataforma de e-learning, LEFT JOIN permite obtener una lista de estudiantes registrados, junto con los cursos inscritos. Aun si no están matriculados, su nombre aparecerá en el reporte.

Buenas prácticas y errores comunes

Buenas prácticas

- Revisar cuántos registros NULL aparecen en los resultados, para detectar claves sin coincidencia.
- Comprobar que el campo de unión tenga índices para mejorar el rendimiento.
- Verificar duplicación de registros por relaciones uno-a-muchos mal controladas.

Errores comunes

- Malinterpretar los NULL: pueden significar ausencia de relación, no necesariamente error.
- Olvidar filtrar con IS NULL cuando se desea detectar claves huérfanas.
- Repetir columnas innecesarias, generando resultados confusos.
- Olvidar que el orden de las tablas afecta el resultado.

Comparación entre INNER JOIN y LEFT JOIN

Característica	INNER JOIN	LEFT JOIN
¿Incluye no coincidencias?	No	Sí (con NULL)
Tamaño del resultado	Igual o menor que la tabla con menos registros	Igual o mayor que la tabla izquierda
Útil para...	Reportes cerrados o análisis de actividad	Listados completos, auditorías

Tabla 1. Comparación entre INNER JOIN y LEFT JOIN

Fuente: Desafío Latam

LEFT JOIN es especialmente útil cuando se necesita una visión completa, incluyendo registros sin relación aparente. Permite validar cobertura, detectar faltantes y construir reportes amplios que incluyan usuarios inactivos, productos sin ventas o servicios no utilizados.

Actividad práctica autónoma:

**Integrando información de
clientes y órdenes**



Objetivo de la actividad

Aplicar combinaciones INNER JOIN y LEFT JOIN sobre un modelo de datos simulado para integrar información dispersa entre clientes y órdenes de servicio, identificando diferencias de resultado y analizando implicancias de negocio.



Situación problema

Formas parte del equipo de datos de una empresa de servicios que desea evaluar el comportamiento de sus clientes.

Tienes dos fuentes de información: una tabla de clientes y una de órdenes emitidas.

Se desea conocer quiénes han realizado órdenes, quiénes no, y cuántas.

Tu tarea es integrar ambos conjuntos de datos para responder a esas preguntas, usando combinaciones apropiadas.



Código base

```
CREATE TABLE clientes (  
    id_cliente INTEGER PRIMARY KEY,  
    nombre TEXT,  
    email TEXT  
);  
  
CREATE TABLE ordenes_servicio (  
    id_orden INTEGER PRIMARY KEY,  
    id_cliente INTEGER,  
    fecha_orden DATE,  
    monto REAL  
);  
  
INSERT INTO clientes VALUES  
    (1, 'Ana Ruiz', 'ana@example.com'),  
    (2, 'Carlos Soto', 'carlos@example.com'),  
    (3, 'Lucía Peña', 'lucia@example.com');  
  
INSERT INTO ordenes_servicio VALUES  
    (101, 1, '2024-05-10', 120.0),  
    (102, 1, '2024-06-11', 135.5),  
    (103, 2, '2024-06-12', 99.9);
```



Instrucciones

Ejecutar las siguientes consultas:

```
-- Consulta A: INNER JOIN
SELECT
  c.nombre,
  o.fecha_orden,
  o.monto
FROM clientes c
INNER JOIN ordenes_servicio o ON c.id_cliente = o.id_cliente;

-- Consulta B: LEFT JOIN
SELECT
  c.nombre,
  o.fecha_orden,
  o.monto
FROM clientes c
LEFT JOIN ordenes_servicio o ON c.id_cliente = o.id_cliente;

-- Consulta C: clientes sin órdenes
SELECT c.nombre
FROM clientes c
LEFT JOIN ordenes_servicio o ON c.id_cliente = o.id_cliente
WHERE o.id_orden IS NULL;
```

¿Qué registros aparecen en cada caso?

¿Qué tipo de JOIN es más útil según el objetivo del informe?

¿Cómo interpretar los NULL?

Actividad práctica autónoma:

Uniendo tablas para informes de clientes



Objetivo de la actividad

Aplicar combinaciones INNER JOIN y LEFT JOIN de manera autónoma para integrar información desde múltiples tablas relacionadas, con el fin de construir reportes que identifiquen actividad y ausencia de actividad de clientes en un sistema de órdenes de servicio.



Contexto

Eres analista de datos en una empresa de soporte técnico que desea optimizar sus campañas de retención de clientes.

Para ello, necesitas generar un informe que muestre qué clientes realizaron órdenes en el último trimestre y cuáles no.

Los datos provienen de dos tablas separadas: una que contiene información general de los clientes (clientes) y otra que registra las órdenes de servicio (ordenes_servicio).

Tu tarea es construir tres consultas SQL diferentes, analizar los resultados y justificar cuál usarías en distintos escenarios de análisis.



Instrucciones

Crea las tablas base y carga los datos:

```
CREATE TABLE clientes (  
  id_cliente INTEGER PRIMARY KEY,  
  nombre TEXT,  
  email TEXT  
);  
  
CREATE TABLE ordenes_servicio (  
  id_orden INTEGER PRIMARY KEY,  
  id_cliente INTEGER,  
  fecha_orden DATE,  
  monto REAL  
);  
  
INSERT INTO clientes VALUES  
  (1, 'Valeria Fuentes', 'valeria@correo.com'),  
  (2, 'Ricardo Palma', 'ricardo@correo.com'),  
  (3, 'Javiera Olivares', 'javiera@correo.com'),  
  (4, 'Mauricio Vidal', 'mauricio@correo.com');  
  
INSERT INTO ordenes_servicio VALUES  
  (201, 1, '2024-04-10', 89.0),  
  (202, 1, '2024-06-15', 112.5),  
  (203, 2, '2024-07-05', 76.3);
```



Consultas a ejecutar

Ejecuta las siguientes consultas y analiza los resultados:

1

Consulta con INNER JOIN:

```
SELECT
    c.nombre,
    o.fecha_orden,
    o.monto
FROM clientes c
INNER JOIN ordenes_servicio o ON
c.id_cliente = o.id_cliente;
```

2

Consulta con LEFT JOIN:

```
SELECT
    c.nombre,
    o.fecha_orden,
    o.monto
FROM clientes c
LEFT JOIN ordenes_servicio o ON
c.id_cliente = o.id_cliente;
```

3

Consulta de clientes sin órdenes:

```
SELECT c.nombre
FROM clientes c
LEFT JOIN ordenes_servicio o ON
c.id_cliente = o.id_cliente
WHERE o.id_orden IS NULL;
```

Preguntas para el documento de entrega:

- ¿Cuántos clientes realizaron órdenes?
- ¿Qué diferencia observas entre el resultado del INNER JOIN y el LEFT JOIN?
- ¿Qué clientes no han emitido órdenes? ¿Qué acción tomarías con ellos?
- ¿En qué casos usarías un LEFT JOIN en vez de un INNER JOIN?
- ¿Cuál de estas consultas usarías para construir un reporte de actividad mensual de clientes activos e inactivos? ¿Por qué?

Resumen



Comprendimos la importancia de las combinaciones entre tablas para construir consultas más completas y útiles.



Aprendimos a aplicar INNER JOIN para obtener coincidencias exactas entre tablas relacionadas.



Aplicamos LEFT JOIN para conservar información de una tabla incluso cuando no hay coincidencias en la otra.



Identificamos las diferencias clave entre ambos tipos de combinación y su impacto en los resultados.



Analizamos situaciones reales donde cada tipo de JOIN es más conveniente, evaluando los efectos de los valores nulos.



Próxima sesión...

En la siguiente sesión continuaremos profundizando en las combinaciones de tablas, abordando RIGHT JOIN y FULL OUTER JOIN.

Veremos cómo estas combinaciones permiten acceder a información que se encuentra exclusivamente en una u otra tabla, facilitando diagnósticos más completos, análisis de cobertura de registros y auditorías de integridad de datos en contextos reales.

Preguntas de cierre

¿Cuál es la diferencia fundamental entre INNER JOIN y LEFT JOIN?

¿En qué casos utilizarías un LEFT JOIN sobre un INNER JOIN?

¿Qué implicancias tiene encontrar muchos valores NULL en el resultado de un LEFT JOIN?

¿Cómo podrías usar un JOIN para detectar registros huérfanos o inactivos?

¿Qué errores comunes pueden surgir al unir tablas y cómo prevenirlos?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2022). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>
- Oracle. (2024). SQL Language Reference. <https://docs.oracle.com/en/database/>
- Gobierno de Chile. (s.f.). Datos Abiertos. <https://datos.gob.cl>